

TensorForge 2.0 Phase 2 — Video Script (12–13 min)

Oct 6, 2026 · @osal_geesara

Team Nexio, University of Moratuwa: H.K.Osal Geesara, Sahas Samudhitha, Chamalka Heshani, Poojani Danulya, Rashmi Abeysekara

A 12–13 minute walkthrough of the solution, architecture and evaluation, for a YouTube unlisted upload. Each segment lists what is on screen and what to say; timings include demo actions and add up to about 12:30, under the 15-minute limit. Spoken text alone is about 9½ minutes.

Before recording

Set everything up first so the recording never waits on a page load. **Never show the real API key:** use `test123` locally, and run hosted commands with the key loaded from `.env` so it never appears on screen.

- ☐ Browser tab 1: the GitHub repo README (architecture diagram visible)
- ☐ Browser tab 2: `https://13-232-241-157.sslip.io/demo/` (Single ticket tab)
- ☐ Browser tab 3: `https://13-232-241-157.sslip.io/docs`
- ☐ Image viewer with `reports/figures/`: `eda_category_counts.png`, `eda_language_counts.png`, `exp_progress.png`, `exp_xlmr_training.png`, `eval_confusion_matrix.png`, `eval_accuracy_by_language.png`, `eval_calibration.png`
- ☐ A labelled CSV for the demo upload (for example 200 rows of `data/validation.csv`)
- ☐ Terminal 1: project folder, `.venv` ready, `.env` loaded with `set -a && . ./env && set +a`
- ☐ Terminal 2: the Docker image built (`docker build -t tensorforge .`)
- ☐ VS Code open on `app/main.py`, `app/model.py`, `notebooks/02_transformer_colab.ipynb`
- ☐ Screen at 1920×1080, terminal font 16+, notifications off

0:00–1:00 · Introduction and the problem

On screen	Say
Demo page, a ticket already classified	"Hi, we're Team Nexio from the University of Moratuwa, and this is our TensorForge 2.0 Phase 2 submission: a system that reads a RideEat support ticket and routes it to the right team, in English, Sinhala, Tamil, Singlish or Tanglish."
README top: links table	"For each ticket we predict the main category, a second issue if there is one, and whether it is urgent. The team then follows from a fixed table in the specification, so we never learn it, we just apply it."
README results table	"Our own fine-tuned model gets 85.3% category accuracy on validation, against 60.4% for our baseline. No LLM is called anywhere."
README section list	"In the next few minutes: the data, how we trained and improved the model, the results, the architecture, a live demo, and how we tested it."

1:00–2:10 · The data and what EDA showed

On screen	Say
<code>eda_category_counts.png</code>	"The organisers gave us 4,000 training tickets and 800 for validation. The classes are imbalanced: payment refunds have 720 tickets, spam only 120, a six-fold gap. So we used class weights everywhere."
<code>eda_language_counts.png</code>	"The language mix is the real challenge: 35% English, 25% Singlish, 20% Sinhala script, 15% Tamil, plus Tanglish and mixed. The data has a language column, but the API never sends it, so the model has to work without it."

On screen	Say
<code>reports/eda_summary.md</code> , urgent table	"Only 10% of tickets are urgent, but 75% of safety tickets are. And 10% carry a secondary issue, like a refund request inside a late-delivery complaint."
<code>reports/eda_summary.md</code> , nearest-neighbour table	"One finding changed how we evaluate. Training tickets have close paraphrases of each other: a ticket's most similar neighbour shares its category 82% of the time. For validation tickets it's only 31%. Keep that in mind, it matters in a minute."

2:10–4:30 · How the model was trained, validated and improved

The judges look hardest at this part. Show real files, not slides.

On screen	Say
<code>notebooks/01_baseline.py</code>	"We started with a classical baseline: TF-IDF on character and word n-grams, logistic regression with class weights, one model per output. Validation accuracy: 60.4%."
<code>reports/experiments.csv</code>	"Then we logged every experiment in this file, all reproducible from <code>notebooks/04_experiments.py</code> with a fixed seed: words versus characters, class weights, regularisation, a linear SVM, and removing channel and subject."
<code>exp_progress.png</code>	"Character n-grams beat words, 63% against 53%, because Singlish and Tanglish spelling varies so much. The best classical model was a linear SVM at 64.6%. Then a ceiling."
<code>experiments.csv</code> , cv5 column	"Here is the trap. In 5-fold cross-validation on the training set, all these models score about 98%. On validation, 53 to 65%. That's the paraphrase effect from the EDA: random folds just find the near-copy. So we compared every model on held-out validation only."

On screen	Say
<code>02_transformer_colab.ipynb</code> , settings + model cell	"To get past the ceiling we fine-tuned XLM-RoBERTa base, which was pretrained on 100 languages including Sinhala and Tamil. One encoder, three small heads: category, secondary and urgency, trained together with class weights for the rare classes."
<code>exp_xlmr_training.png</code>	"We trained eight epochs with two seeds. Both reach about 85% by epoch five or six and then flatten. A higher learning rate was unstable, 83% then 78% on identical runs, so we used 2e-5. We kept seed 42, epoch 6."
Notebook section 9	"For serving we export to ONNX so the server needs no PyTorch or GPU. We tried int8 compression, but it cost 3 to 4 points, so we kept full precision. Latency isn't scored; accuracy is."
<code>eval_calibration.png</code>	"Last improvement: the raw model said 96% confidence on average but was right 85% of the time. The spec scores calibration, so we fitted one temperature value, 1.75. Now 80% confidence really means about 80%. Calibration error went from 0.106 to 0.025, and no prediction changed."

4:30–5:45 · Evaluation results

On screen	Say
<code>reports/metrics.json</code> or the README results table	"Final results on the 800 validation tickets, computed through the same code the API uses: category accuracy 85.3% against 60.4%, macro-F1 0.85, urgent F1 0.97 with 98.8% of urgent tickets caught, and all three outputs right on 83% of tickets."

On screen	Say
<code>eval_confusion_matrix.png</code>	"The confusion matrix shows where it still goes wrong. The biggest error: 25 vague refund requests sent to general inquiry. App problems are sometimes read as delivery delays. Safety tickets, the most important, are 30 of 32 correct."
<code>eval_accuracy_by_language.png</code>	"By language, it's strong in native scripts: Sinhala 86%, Tamil 88%, English 88%. Singlish is the weakest at 79%, and interestingly the only language where the simple baseline does better. That's our clearest next step."
<code>eval_calibration.png</code>	"And the confidence is honest: the line follows the diagonal. Tickets under 50% are flagged for human review: about 5% of tickets, and only 19% of those would have been right."

5:45–7:30 · Architecture and API compliance

On screen	Say
README architecture diagram	"Here's the system. Requests arrive over HTTPS at Caddy on an AWS server, which forwards to our Docker container. Inside, FastAPI checks every request in the exact order the spec demands: API key, content type, size, JSON, then validation. Only then does it reach the model."
<code>app/main.py</code> , <code>check_auth</code>	"The key is read only from the <code>API_KEY</code> environment variable, never hard-coded, and compared in constant time. Both <code>X-API-Key</code> and <code>Bearer</code> work. If no key is configured, everything returns 401: the service never runs open."
<code>app/model.py</code> , <code>decide</code> and <code>TEAMS</code>	"After the model, fixed rules: the team comes from the spec's table, the secondary category must differ from the primary, and spam is never urgent. We apply these rather than learn them, so they can't be wrong."

On screen	Say
<code>/docs</code> page, the endpoint list	"All seven endpoints from the contract: health, predict, batch of up to 100, and the async jobs for up to 5,000 tickets: submit, poll, page through results, delete."
<code>app/jobs.py</code>	"Jobs live in SQLite, so polling is reliable. If the container restarts mid-job, the job ends as failed with code interrupted, never lost silently. And live predict requests always run before job work, so the API stays fast during a big job."
<code>api/error_response.schema.json</code>	"Every error uses the spec's error format, including unknown routes, wrong methods and malformed JSON. Bad input never produces a 500."

7:30–9:30 · Live demo: website and hosted API

On screen	Say
Demo page, type a Singlish ticket: <i>bro mage order eka hour ekakata wada late, driver call ganne na. refund ekak denna</i>	"This is the live demo. A Singlish ticket: the order is an hour late and they want a refund. Classify: delivery delay, routed to Delivery Operations, not urgent, with its confidence."
Switch to Call, paste: <i>customer: the driver took a wrong turn and he's not stopping the car, i am scared please</i>	"A call transcript about an unsafe driver: safety, Trust & Safety, flagged urgent."
Type a Tamil or Sinhala ticket	"Native script works too, with no translation step."
Batch file tab, upload the labelled CSV	"For bulk work, upload a CSV. With labels, we get accuracy, how many urgent tickets were caught, accuracy per category and a confusion matrix, all live."
Say while on the demo	"The page never holds our API key. It calls small server-side demo routes that run the same model at low priority, so visitors can't slow down the judges."

On screen	Say
Terminal: <code>curl -s -X POST https://13-232-241-157.sslip.io/predict -H 'Content-Type: application/json' -d '{"channel":"chat","text":"hi"}'</code>	"The real API is protected: no key, 401, in the spec's error format."
Terminal: the same with <code>-H "X-API-Key: \$API_KEY"</code>	"With the key, a prediction, with every field the schema requires."

9:30–11:30 · Docker offline and testing

The smoke test takes one to two minutes against the hosted URL. Start it at the beginning of this segment, or speed up the recording while it runs.

On screen	Say
Dockerfile	"The model is copied into the image at build time, so nothing is downloaded at runtime: slim Python base, non-root user, health check."
Terminal 2: <code>docker run --rm --network none --cpus 2 --memory 4g -e API_KEY=test123 tensorforge</code>	"Here it runs with networking switched off completely, on the reference limits of 2 CPUs and 4 GB. It's healthy in about 4 seconds, well inside the 120 allowed, and uses about 1.5 GB."
Terminal 1: <code>.venv/bin/python -m pytest -q</code>	"64 automated tests check every response, including every error, against the official JSON Schemas: auth on every endpoint, the edge cases, batch order, the full job lifecycle, even a restart in the middle of a job."
Terminal 1: <code>.venv/bin/python scripts/smoke_test.py https://13-232-241-157.sslip.io</code>	"And this smoke test runs about 250 checks against the live server, including a real async job on validation tickets. 252 out of 252 pass. It also caught a real bug before submission: oversized uploads got a dropped connection instead of a clean 413. Fixed and redeployed."

11:30–12:30 · Findings and close

On screen	Say
<code>reports/README.md</code> , key findings	"Three lessons. First, a fine-tuned multilingual encoder was the decisive step: 20 points over any classical model, in every language. Second, measure honestly: random cross-validation said 98%, held-out validation said 60%. Third, calibration is cheap and matters: one number made the confidence trustworthy."
Report, limitations	"Our limits: Singlish is the weakest language, vague refunds get mistaken for general questions, and we chose the checkpoint on validation, so the hidden test score may be a little lower. Next we'd retrain on train plus validation and add a character-level model for Singlish."
Demo page	"Everything is reproducible from the repository: the data analysis, every experiment, the training notebook, the evaluation and the tests. The API is live over HTTPS, and the demo is open to try. From all of Team Nexio, thank you."